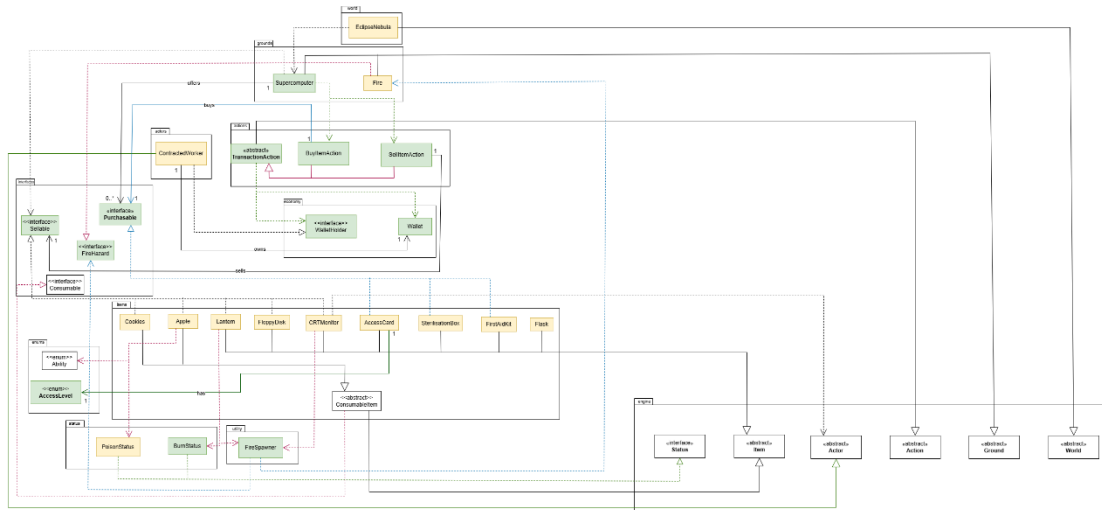


FIT2099 Assignment 2 REQ1

Design Documentation

Design Diagrams

REQ 1 UML Class Diagram:



Design Rationale

REQ1 Design Rationale: The Economy of Eclipse Nebula

For Requirement 1, I designed an economy system that allows workers to buy and sell items through the Supercomputer. The main classes added for this requirement are Wallet, WalletHolder, Supercomputer, TransactionAction, BuyItemAction, SellItemAction, Purchasable, Sellable, FireHazard, FireSpawner, BurnStatus, and AccessLevel. Several existing item classes were also revised, including Apple, Cookies, Lantern, FloppyDisk, CRTMonitor, FirstAidKit, SterilisationBox, and AccessCard.

The Wallet class is responsible for storing and managing credits. It enforces the maximum wallet capacity of 1,000 credits and prevents credits from becoming negative. This keeps credit-related rules inside one cohesive class instead of spreading wallet validation across multiple actions. ContractedWorker implements the WalletHolder interface, allowing transaction actions to access a wallet through an abstraction rather than depending directly on the concrete ContractedWorker class. This reduces coupling and avoids checking the actor's concrete type.

The buying and selling actions are handled by BuyItemAction and SellItemAction, which both extend the abstract TransactionAction class. TransactionAction centralises shared wallet access logic, reducing duplicated code and supporting the DRY principle. BuyItemAction depends on the Purchasable interface, while SellItemAction depends on the Sellable interface. This means the actions do not need to know the

concrete item type, such as Apple, Lantern, or AccessCard. Instead, each item is responsible for its own price and transaction effect. This improves cohesion because item-specific behaviour is kept inside the item that owns that behaviour.

I chose to use Purchasable and Sellable interfaces to avoid the type-checking problem from Assignment 1. In Assignment 1, using instanceof was identified as a major design issue because it made the code depend on concrete classes. In this design, the system asks whether an item has a selling or purchasing capability instead of asking whether it is a specific item class. This better supports polymorphism, the Open-Closed Principle, and the Dependency Inversion Principle. For example, adding a new sellable item in the future would mainly require the new item to implement Sellable, rather than modifying SellItemAction with additional conditional logic.

I considered placing all buying and selling rules directly inside Supercomputer. This would have made the implementation simpler at first, but it would also make Supercomputer responsible for too many item-specific rules, such as Apple poisoning, Lantern fire spawning, and Access Card hidden fees. This would reduce cohesion and create a large class that is harder to maintain. Instead, I placed item-specific purchase and sale effects inside the relevant item classes through Purchasable and Sellable. The benefit is lower coupling between the transaction actions and concrete item classes. The trade-off is that there are more interfaces and more collaboration between classes, but this makes the design easier to extend.

I also considered passing the purchasable stock list into Supercomputer from EclipseNebula to reduce coupling further. However, I chose to let Supercomputer initialise its own purchasable catalogue because the Supercomputer is the trading terminal described in the requirement. Keeping the catalogue inside Supercomputer makes the shop configuration simple, centralised, and easier to understand. The trade-off is that Supercomputer still depends on concrete purchasable item classes such as FirstAidKit, SterilisationBox, and AccessCard. However, the item-specific purchase effects are still handled through the Purchasable interface, so Supercomputer does not need to know how each item behaves internally.

For sale effects involving fire, I introduced FireSpawner and FireHazard. FireSpawner is responsible for spawning fire around a location, while FireHazard allows the system to check whether a ground already has an active fire effect without checking directly for the concrete Fire class. This avoids concrete type checking and keeps fire-spawning logic reusable for both Lantern and CRTMonitor. BurnStatus and PoisonStatus also print remaining turns, which improves player feedback for ongoing effects and addresses the Assignment 1 issue where ongoing effects did not clearly communicate their remaining duration.

Overall, this design aims to keep responsibilities separated: Supercomputer provides trading actions, TransactionAction handles shared transaction behaviour, Wallet manages credits, and individual items define their own buying or selling effects. Compared with a design based on large conditional statements inside one class, this solution has lower coupling, better cohesion, less duplicated logic, and stronger alignment with SOLID principles. The main cost is a larger number of small interfaces and classes, but this trade-off is acceptable because the requirement contains many different item-specific transaction rules.